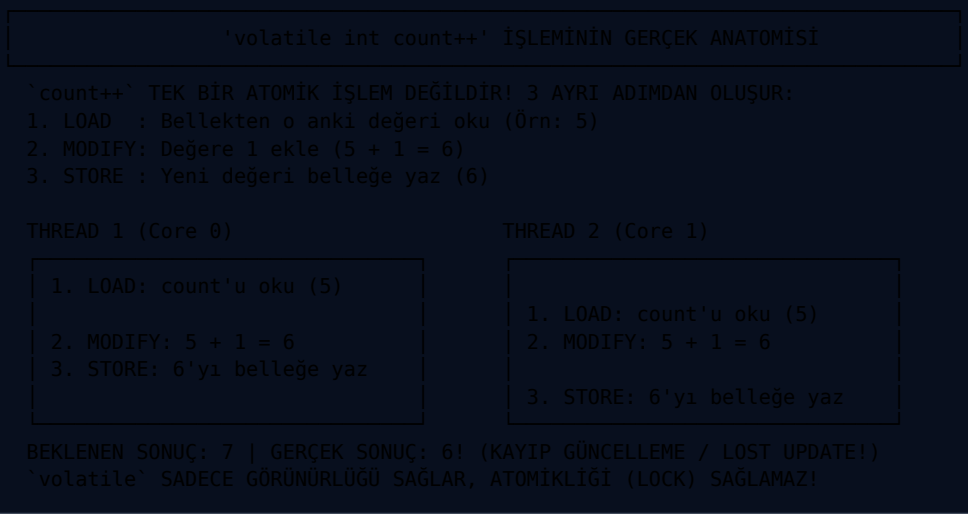
7

BÖLÜM: VOLATILE ANAHTAR KELIMESI, ATOMIKLIK & GÖRÜNÜRLÜK | MODULE: THE VOLATILE KEYWORD, ATOMICITY & VISIBILITY

Görünürlük (Visibility) vs Atomiklik (Atomicity): 'volatile int counter++' Neden Çöker?

Visibility vs Atomicity: Why 'volatile int counter++' is a Silent Catastrophe

⚡ Bellek & Eşzamanlılık Akış Şeması ⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu) 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir kütüphanedeki tahtaya tebeşirle sayı yazıldığını düşünün. 'volatile' demek, tahtanın üzerine süper parlak bir spot ışığı yakmak gibidir; tahtada ne yazıyorsa bütün oda anında görür (Görünürlük)! Ama iki kişi aynı anda tahtaya koşup 'Mevcut sayıyı silip 1 fazlasını yazayım' derse ne olur? İkisi de aynı anda '5' sayısını görür, ikisi de kafasında 6 hesaplar, ikisi de tahtaya 6 yazar! 2 kere artması gereken sayı sadece 1 kere artar! Spot ışığı (volatile) insanların birbirinin eline çarpmasını (Atomiklik) engellemez!

🇬🇧 ENGLISH (ELI5)

Imagine a shared blackboard in an office. Declaring a variable `volatile` is like installing a 10,000-lumen spotlight directly onto the board: the instant a number is inscribed, every engineer across the room sees it immediately (Visibility). But what happens when two engineers run to the board at the same second to increment the number? Both observe '5', both mentally calculate '5 + 1 = 6', and both scribble '6' over the board! The count increments once instead of twice! Spotlights guarantee sight, not turn-taking (Atomicity)!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Read-Modify-Write Döngüsü ve volatile Sınırları: 1) Görünürlük Garantisi: `volatile` bir değişkene yazıldığında, CPU yerel Store Buffer'ı ana belleğe boşaltır (Flush). Okunduğunda ise CPU yerel cache'i geçersiz kılar (Invalidate) ve RAM/L3'ten taze veri çeker. 2) Atomiklik İhlali: Java bytecode seviyesinde `count++` işlemi: `GETFIELD`, `ICONST_1`, `IADD`, `PUTFIELD` olmak üzere 4 talimata ayrılır. İki thread bu talimatları iç içe (interleaved) çalıştırdığında 'Lost Update' meydana gelir. 3) Kural: `volatile` yalnızca ve yalnızca yazılan yeni değer eski değerden tamamen bağımsızsa (örneğin bir boolean bayrak: `running = false`) güvenlidir. Eğer yeni değer eski değere bağlıysa (`count = count + 1`), `volatile` yetersizdir; `AtomicInteger` veya `synchronized` zorunludur.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Dissecting the Read-Modify-Write Hazard: 1) Hardware Visibility Semantics: A `volatile` write issues a memory barrier draining the local store buffer to the cache coherency fabric. A `volatile` read bypasses stale L1 entries. 2) Bytecode Non-Atomicity: The expression `count++` compiles into non-atomic instructions: `getfield`, `iconst_1`, `iadd`, `putfield`. Context switches occurring between `getfield` and `putfield` produce severe data races (Lost Updates). 3) The Axiom of Volatile: `volatile` is thread-safe IF AND ONLY IF state mutations do not depend on current state (e.g. status flags: `volatile boolean shutdownRequested`). When reads dictate writes, CAS primitives (`AtomicInteger`) or mutual exclusion are non-negotiable.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: Volatile Atomiklik İllüzyonu ve Kayıp Güncelleme Kanıtı
public class VolatileBrokenCounter {
    private volatile int count = 0;

    public void increment() {
        count++; // TEHLİKELİ! Atomik değildir!
    }

    void main() throws InterruptedException {
        var demo = new VolatileBrokenCounter();
        int threadCount = 10;
        int incrementsPerThread = 1000;

        Thread[] threads = new Thread[threadCount];
        for (int i = 0; i < threadCount; i++) {
            threads[i] = Thread.ofPlatform().start(() -> {
                for (int j = 0; j < incrementsPerThread; j++) {
                    demo.increment();
                }
            });
        }

        for (Thread t : threads) t.join();

        IO.println("Beklenen Sayaç: " + (threadCount * incrementsPerThread)); // 10,000
        IO.println("Gerçek Sayaç  : " + demo.count); // Çoğunlukla 9,200 civarı çıkar!
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakat Klasiği: 'Volatile anahtar kelimesi senkronizasyon için ne zaman yeterlidir?' Yanıt: 'Sadece tek bir thread yazıp diğer tüm thread'ler okurken (Single-Writer Multiple-Reader) veya değişken sadece durum bildiren bir bayraksa (`volatile boolean stopped`). Birden fazla thread yazıyorsa volatile tek başına ASLA yetmez!'

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Top Interview Screen: 'When is volatile sufficient for thread safety?' Senior Answer: 'Exclusively in Single-Writer / Multiple-Reader topologies, or for state flag publication (`volatile boolean cancelled`). If multiple concurrent threads execute read-modify-write operations, volatile guarantees zero safety without locks or CAS operations.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- volatile sadece görünürlüğü (visibility) sağlar; atomiklik (atomicity) sağlamaz.
- count++ arka planda 3 ayrı işlem olduğu için lost update hatasına yol açar.
- Değişkene birden fazla thread yazıyorsa Atomic sınıfları veya kilitler zorunludur.

🎯 Key Takeaways for the Interview

- volatile guarantees memory visibility; it provides zero transactional atomicity.
- count++ encompasses 3 discrete CPU instructions, suffering lost updates under concurrency.
- When multiple threads mutate shared state, Atomic types or mutexes are mandatory.